

Facultad de
Ciencias Exactas,
Ingeniería y Agrimensura



Trabajo práctico I

Aprendizaje automático II

Redes Neuronales Feed Forward

Año 2026 - 1° Semestre

Integrantes: Juana Chies Doumecq (C-7554/1), Sebastian Palacio (P-5357/1).

Problema 1

Objetivo

El objetivo fue construir un modelo de red neuronal capaz de predecir si un paciente tiene alto riesgo de sufrir un accidente cerebrovascular (ACV) a partir de datos clínicos y demográficos. Se trabajó con el dataset healthcare-dataset-stroke-data.csv, que contiene variables como edad, nivel de glucosa, índice de masa corporal, hipertensión, enfermedad cardíaca, estado civil, tipo de trabajo, tipo de residencia y estado de fumador.

Procedimiento

Exploración y Análisis Exploratorio de Datos (EDA)

- Inspección de la distribución de cada variable mediante histogramas y gráficos de caja.
- Análisis del desbalance de clases: se detectó que la clase positiva (stroke=1) representa apenas el 4.87% de los datos de entrenamiento, configurando un escenario de desbalance extremo.
- Análisis de correlaciones entre variables numéricas y la variable objetivo mediante el coeficiente de correlación biserial puntual.
- Pairplot de variables numéricas para visualizar la separabilidad entre clases. Se confirmó que la edad (age) es la variable con mayor poder discriminativo.

Preprocesamiento

- Eliminación de la columna id (identificador no informativo) y del único registro con género 'Other'.
- Imputación de valores faltantes en la columna bmi mediante KNN Imputer (k=5), preservando la estructura local de los datos.
- Codificación de variables categóricas mediante One-Hot Encoding para las variables nominales (gender, work_type, Residence_type, smoking_status) y codificación binaria para ever_married.
- División del dataset en conjuntos de entrenamiento (80%) y test (20%) con estratificación por la variable objetivo.
- Normalización de features numéricas con StandardScaler, ajustado exclusivamente sobre el conjunto de entrenamiento.
- Separación del conjunto de entrenamiento en train real (75%) y validación (25%), también con estratificación.
- Aplicación de SMOTE (Synthetic Minority Oversampling Technique) únicamente sobre el conjunto de entrenamiento para balancear las clases (50%/50%), dejando los conjuntos de validación y test con la distribución real.

Diseño e Implementación de la Red Neuronal: se diseñó un Perceptrón Multicapa (MLP) implementado en PyTorch con la siguiente configuración:

- Capa de entrada: dimensión igual al número de features (20 variables tras el preprocesamiento).
- Primera capa oculta: Linear(20→64) → BatchNorm1d → ReLU → Dropout(0.2).

- Segunda capa oculta: Linear(64→32) → BatchNorm1d → ReLU → Dropout(0.2).
- Capa de salida: Linear(32→1) sin activación (logit crudo).

Entrenamiento: el modelo fue entrenado con los siguientes hiperparámetros y estrategias:

- Función de pérdida: BCEWithLogitsLoss (combina sigmoide y entropía cruzada binaria de forma numéricamente estable).
- Optimizador: Adam con learning rate inicial de $1e-3$ y weight decay de $1e-4$.
- Scheduler: ReduceLROnPlateau en modo max sobre el Recall de validación; reduce el learning rate a la mitad si no hay mejora en 10 épocas.
- Early Stopping: paciencia de 30 épocas monitoreando el Recall de validación.
- Búsqueda de threshold óptimo en cada época sobre el conjunto de validación.
- Entrenamiento hasta 200 épocas con batch size de 32.

Evaluación

La evaluación del modelo se realizó sobre el conjunto de test (distribución real, sin SMOTE), utilizando el threshold óptimo encontrado en validación. Las métricas reportadas incluyeron Recall, Accuracy, ROC-AUC, matriz de confusión y curva ROC. Los resultados finales fueron:

- Verdaderos Negativos (TN): 701 pacientes sanos identificados correctamente.
- Falsos Positivos (FP): 271 pacientes sanos clasificados como de riesgo.
- Falsos Negativos (FN): 12 pacientes con riesgo no detectados.
- Verdaderos Positivos (TP): 38 pacientes con riesgo identificados correctamente.

El modelo priorizó el Recall sobre la clase positiva, lo que resulta adecuado para un contexto médico donde no detectar un caso real de ACV tiene consecuencias más graves que un falso positivo.

Decisiones de Diseño

- Elección de la Métrica Principal: Recall

Se priorizó el Recall (sensibilidad) como métrica principal en lugar de la Accuracy. Por un lado, el desbalance extremo (~95% clase negativa) hace que la Accuracy sea engañosa (un modelo trivial que siempre predice 0 tendría 95% de accuracy sin aprender nada útil). Por otro lado, en el dominio médico, un falso negativo (no detectar un paciente con riesgo real de ACV) tiene consecuencias mucho más graves que un falso positivo (alertar innecesariamente a un paciente sano).

- Estrategia de Balanceo: SMOTE solo en Train

Se eligió SMOTE en lugar de otras alternativas como class weights en la función de pérdida o simple oversampling aleatorio. SMOTE genera muestras sintéticas interpolando entre puntos reales del espacio de features, lo que enriquece la representación de la clase minoritaria sin simplemente repetir instancias. Es fundamental que SMOTE se aplique únicamente sobre el conjunto de entrenamiento y no sobre validación ni test, para garantizar que la evaluación refleje la distribución real del problema y no esté artificialmente inflada.

- Arquitectura Decreciente del MLP (64 → 32)

Se diseñó un MLP con dos capas ocultas de tamaño decreciente (64 → 32 neuronas). Esta arquitectura fuerza al modelo a comprimir la información en representaciones progresivamente

más abstractas, operando como un cuello de botella controlado. Una arquitectura más profunda o más ancha podría haber incurrido en mayor overfitting dada la pequeña cantidad de datos reales disponibles.

- BatchNorm y Dropout

Después de cada capa lineal se aplicó BatchNorm1d seguido de Dropout(0.2). El BatchNorm estabiliza las activaciones a lo largo del entrenamiento, acelerando la convergencia y reduciendo la sensibilidad al learning rate. El Dropout actúa como regularización implícita al desactivar neuronas aleatoriamente durante el entrenamiento, reduciendo la co-dependencia entre unidades. La tasa de Dropout de 0.2 (versus 0.3 en versiones previas) fue elegida para balancear regularización con capacidad de aprendizaje.

- BCEWithLogitsLoss y Logits Crudos en la Salida

La capa de salida produce un logit crudo (sin sigmoide). La función BCEWithLogitsLoss combina internamente la sigmoide y la entropía cruzada binaria en una sola operación numéricamente estable, evitando problemas de saturación que ocurrirían si se aplicara la sigmoide explícitamente antes de la pérdida. Durante la evaluación, se aplica torch.sigmoid() para convertir los logits en probabilidades interpretables.

- Early Stopping sobre Recall de Validación

El Early Stopping se configuró para monitorear el Recall de validación (y no la pérdida de validación), con una paciencia de 30 épocas. Esto es coherente con la elección del Recall como métrica principal: se guardan los pesos del modelo correspondiente al mayor Recall de validación alcanzado, garantizando que el modelo final sea el que mejor detecta casos positivos y no el que minimiza la pérdida promedio.

Desafíos Encontrados

- Desbalance Extremo de Clases: la clase positiva (stroke=1) representa apenas el 4.87% de los datos. Sin ninguna intervención, el modelo tendía a ignorar casi por completo la clase minoritaria, obteniendo alta accuracy pero un Recall prácticamente nulo sobre los casos de ACV.
- Elección de la Métrica de Monitoreo: usar la pérdida o la accuracy como señal de Early Stopping resultó contraproducente, ya que ambas métricas mejoran cuando el modelo simplemente predice más ceros. Pasar a monitorear el Recall de validación resolvió este problema, aunque requirió ajustar la paciencia del scheduler y del Early Stopping de forma coordinada para evitar oscilaciones.
- Trade-off entre Recall y Falsos Positivos: maximizar el Recall inevitablemente incrementa los falsos positivos: el modelo marca como 'de riesgo' a muchos pacientes sanos. En el contexto médico, esto se considera aceptable (es preferible derivar innecesariamente a un paciente sano que no detectar un ACV real), pero requirió explicitar y justificar este trade-off. La matriz de confusión final (271 FP vs 12 FN) refleja esta decisión de diseño.

Conclusiones

En este trabajo se evidenció que, en problemas de clasificación binaria altamente desbalanceados, el rendimiento del modelo depende más de la estrategia de entrenamiento que de la arquitectura, siendo clave optimizar métricas como el Recall, aplicar técnicas de balanceo como SMOTE correctamente y ajustar el threshold de decisión. Asimismo, se destacó la importancia del preprocesamiento (imputación, codificación y normalización) en la calidad del aprendizaje, junto con el uso de regularización (BatchNorm, Dropout y weight decay) para evitar sobreajuste. En contextos médicos, se remarcó la necesidad de utilizar métricas adecuadas como Recall y ROC-AUC en lugar de accuracy, interpretando los resultados según su utilidad clínica. Finalmente, se identificaron limitaciones del dataset y posibles mejoras futuras, como el uso de Focal Loss, validación cruzada estratificada y modelos probabilísticos mejor calibrados.

Problema 2

Objetivo

El objetivo fue construir un modelo de red neuronal capaz de reconocer dígitos escritos a mano (del 0 al 9) a partir de imágenes en escala de grises de 28×28 píxeles. Se trabajó con el dataset MNIST, un benchmark clásico de clasificación multiclase con 60.000 imágenes de entrenamiento y 10.000 de test. Cada imagen se almacena en formato binario IDX y fue cargada manualmente mediante funciones de parsing del formato .idx3-ubyte y .idx1-ubyte.

Procedimiento

Carga y División del Dataset

Los archivos binarios del dataset fueron leídos con funciones propias que parsean el header mágico del formato IDX (magic number 2051 para imágenes y 2049 para etiquetas) y reconstruyen los arrays NumPy con las dimensiones correctas. El conjunto original provee una partición train/test predefinida (60.000/10.000 muestras). Adicionalmente, se separó un subconjunto de validación del entrenamiento para monitorear el comportamiento durante el entrenamiento y aplicar Early Stopping, garantizando que el test permanezca completamente oculto hasta la evaluación final.

Análisis Exploratorio de Datos (EDA)

- Visualización de imágenes: se graficaron 25 imágenes aleatorias del conjunto de entrenamiento con sus etiquetas reales para verificar la correcta carga de los datos y apreciar la variabilidad en la escritura de cada dígito.
- Distribución de clases: se verificó que el dataset está prácticamente balanceado, con aproximadamente 6.000 muestras por clase (el dígito 1 es el más frecuente con 6.742 muestras y el dígito 5 el menos frecuente con 5.421). Este balance hace innecesario aplicar técnicas de sobremuestreo.
- Distribución de píxeles: se analizó el histograma de valores de píxeles, observando la distribución bimodal característica del dataset: la gran mayoría de píxeles son negros (valor 0, fondo) y una minoría son blancos o grises (trazo del dígito).

Preprocesamiento

- Aplanamiento (Flatten): cada imagen de 28×28 píxeles se transformó en un vector de 784 dimensiones para poder ser procesada por capas densas del MLP.
- Normalización por división: los valores de píxel se normalizaron dividiendo por 255.0, llevando cada feature del rango [0, 255] al rango [0.0, 1.0]. Esto evita que gradientes de distintas features tengan magnitudes muy distintas y acelera la convergencia.
- Codificación de etiquetas: las etiquetas se mantuvieron como enteros en el rango [0, 9] (formato uint8), aprovechando que la función de pérdida `sparse_categorical_crossentropy` acepta directamente enteros, eliminando la necesidad de una matriz one-hot de 60.000 × 10 valores.

Diseño e Implementación de la Red Neuronal

Se diseñó un Perceptrón Multicapa (MLP) implementado con TensorFlow/Keras mediante la API Sequential. La arquitectura final consiste en:

- Capa de entrada implícita: 784 neuronas (una por píxel).
- Primera capa oculta: Dense(256, activation='relu') seguida de Dropout(0.3).
- Segunda capa oculta: Dense(128, activation='relu').
- Tercera capa oculta: Dense(64, activation='relu').
- Capa de salida: Dense(10, activation='softmax') — una neurona por clase, con probabilidades que suman 1.

El modelo cuenta con 242.762 parámetros entrenables, concentrados principalmente en la primera capa (200.960 parámetros).

Entrenamiento

El modelo fue compilado con Adam (lr=0.001) y sparse_categorical_crossentropy como función de pérdida, monitoreando la accuracy durante el entrenamiento. Se entrenó hasta 20 épocas con batch size de 128, utilizando el conjunto de validación para monitorear la pérdida y el accuracy en cada época y detectar overfitting.

Evaluación

La evaluación final se realizó sobre el conjunto de test de 10.000 imágenes. Se reportaron las siguientes métricas y análisis:

- Accuracy global en test: 98.24%, confirmando un excelente desempeño general del modelo.
- Reporte por clase (precision, recall, F1-score): todos los dígitos superan el 97% en F1-score. Los dígitos 8 y 9 presentaron los F1 más bajos (0.9767 y 0.9765 respectivamente), posiblemente por su mayor variabilidad morfológica.
- Matriz de confusión: análisis visual de los errores de clasificación entre clases.
- Análisis de errores de alta confianza: visualización de los 5 errores donde el modelo predijo con mayor confianza la clase equivocada, revelando que la mayoría corresponde a dígitos ilegibles o malformados incluso para la percepción humana.

Decisiones de Diseño

- ReLU en Capas Ocultas y Softmax en la Salida

La elección de ReLU para las capas ocultas se justifica por ser computacionalmente simple, no sufrir el problema del gradiente desvanecido (a diferencia de sigmoid o tanh) y ser la función de activación estándar para MLPs modernos. Para la capa de salida, Softmax es la elección natural en clasificación multiclase: convierte los logits crudos en una distribución de probabilidad sobre las 10 clases, donde cada valor representa la probabilidad de que la imagen corresponda a ese dígito y la suma de todas es exactamente 1.

- sparse_categorical_crossentropy como Función de Pérdida

Se eligió sparse_categorical_crossentropy en lugar de categorical_crossentropy. Ambas calculan matemáticamente el mismo valor de pérdida, pero la versión sparse acepta etiquetas como enteros directamente sin necesitar que sean codificadas como vectores one-hot. Esto representa una ventaja práctica significativa en términos de eficiencia de memoria: en lugar de

una matriz de 60.000×10 valores, las etiquetas ocupan simplemente un vector de 60.000 enteros.

- Accuracy como Métrica Principal

A diferencia del Problema 1, en este caso la accuracy es una métrica adecuada y suficiente. El dataset MNIST está prácticamente balanceado (~6.000 muestras por clase), por lo que la accuracy no está sesgada hacia ninguna clase en particular. Además, todos los errores de clasificación tienen el mismo costo operativo: confundir un 3 con un 5 es igual de grave que confundir un 7 con un 1. Estas condiciones hacen que la accuracy sea una representación fiel del desempeño real del modelo.

- Normalización por División entre 255 en lugar de StandardScaler

Se eligió la normalización simple dividiendo por 255 en lugar de aplicar StandardScaler (media=0, std=1). La justificación es que para imágenes en escala de grises todos los píxeles comparten la misma escala y semántica (intensidad de color), por lo que no hay features con rangos radicalmente distintos. La normalización al rango [0, 1] es suficiente para garantizar gradientes bien condicionados, y es una práctica estándar en visión por computadora. StandardScaler calcularía media y desviación por píxel, lo que puede producir valores negativos y no aprovecha ninguna propiedad especial del dominio.

Desafíos Encontrados

- Pérdida de Información Espacial por el Flatten: el aplanamiento de imágenes a vectores de 784 dimensiones supone una pérdida de información estructural: el modelo no sabe que el píxel (14, 14) es el vecino del (14, 15), sino que los trata como dos features independientes cualesquiera. Esto limita la capacidad del modelo para capturar patrones locales como bordes o curvas, que son la clave para reconocer dígitos. El modelo compensa esta limitación aumentando el número de parámetros en la primera capa, pero a un costo computacional mayor que el de una CNN equivalente.
- Identificación de los Dígitos Más Difíciles: los resultados muestran que el 8 (F1=0.9767) y el 9 (F1=0.9765) son los más difíciles de clasificar correctamente. Esto se debe a su mayor variabilidad morfológica: un 9 mal escrito puede parecerse a un 4 o a un 7, y un 8 puede confundirse con un 3 o un 6. La matriz de confusión y el análisis de errores de alta confianza confirmaron que la mayoría de las fallas del modelo no son errores del modelo per se, sino casos genuinamente ambiguos que incluso un humano calificaría con dudas.

Conclusiones

En este problema se demostró que un MLP relativamente simple puede lograr un desempeño muy alto (más del 98% de accuracy) en clasificación multiclase cuando el dataset está balanceado y bien preprocesado, siendo la accuracy una métrica adecuada en este contexto. Se destacó el rol fundamental de la función Softmax para producir distribuciones de probabilidad sobre múltiples clases, y se identificaron limitaciones estructurales del MLP para imágenes, como la falta de invarianza espacial y la alta cantidad de parámetros en la primera capa debido a la entrada aplanada. Aun así, la normalización básica de los píxeles resultó suficiente para un entrenamiento eficiente. Finalmente, la comparación con el problema de

clasificación binaria permitió entender cómo cambian las decisiones de diseño según el contexto, y se propusieron mejoras futuras como el uso de CNNs, data augmentation y estrategias de entrenamiento más robustas.

Problema 3

Objetivo

El objetivo fue construir un modelo de red neuronal capaz de predecir el valor numérico de la rentabilidad (Profit, en dólares) de una transacción comercial a partir de sus características: datos del pedido, del cliente, del producto y del envío. Nos enfrentamos a un problema de regresión: la salida del modelo es un número real continuo, que puede ser positivo (ganancia) o negativo (pérdida). El dataset utilizado es el Superstore Sales Dataset, con 9.994 registros y 21 columnas originales.

Procedimiento

Análisis Exploratorio y Comprensión del Dataset

El proceso comenzó con un análisis exploratorio del dataset. El dataset contiene variables de distintos tipos: fechas (Order Date, Ship Date), identificadores únicos (Row ID, Order ID, Customer ID, Product ID), variables categóricas (Ship Mode, Segment, Region, Category, Sub-Category, State) y variables numéricas continuas (Sales, Quantity, Discount, Profit). El análisis incluyó:

- Visualización de la distribución de Profit: la variable objetivo presenta una distribución asimétrica con outliers significativos en ambos extremos (pérdidas grandes y ganancias grandes), lo que condicionó las decisiones de preprocesamiento y función de pérdida.
- Análisis de correlaciones: se construyó una matriz de correlación y un pairplot entre variables numéricas, revelando una relación positiva notable entre Sales y Profit, y una relación negativa entre Discount y Profit. Los descuentos altos tienden a erosionar los márgenes de ganancia.
- Verificación de datos faltantes: el dataset no presenta valores nulos en ninguna columna, por lo que no fue necesaria ninguna estrategia de imputación.
- Análisis de outliers: se identificaron outliers en Sales y Profit. Se decidió no eliminarlos ya que representan transacciones legítimas del negocio (ventas masivas a empresas o devoluciones con pérdidas grandes). En su lugar, se utilizó RobustScaler para el escalado.

Ingeniería de Características

Dado que las redes neuronales no pueden procesar fechas directamente, se realizó ingeniería de características sobre las columnas temporales:

- Descomposición de fechas: de Order Date y Ship Date se extrajeron el año, mes, día de la semana y trimestre, generando 8 nuevas columnas numéricas (order_year, order_month, order_dow, order_quarter, ship_year, ship_month, ship_dow, ship_quarter).
- Demora de envío: se calculó la variable ship_delay como la diferencia en días entre Ship Date y Order Date, capturando la eficiencia logística de cada pedido.
- Eliminación de columnas no predictivas: se descartaron Row ID, Order ID, Customer ID, Customer Name, Product ID, Product Name (identificadores únicos sin valor predictivo),

City y Postal Code (alta cardinalidad sin valor claro) y Country (constante en todo el dataset, ya que todos los registros son de [EE.UU.](#)).

El dataset resultante quedó con 19 columnas: 6 categóricas y 13 numéricas.

División del Dataset

Se realizó una división en tres conjuntos antes de cualquier transformación de preprocesamiento para evitar data leakage: train (64%), validación (16%) y test (20%). El conjunto de validación se utilizó durante el entrenamiento para monitorear el loss y aplicar Early Stopping. El test permaneció completamente oculto hasta la evaluación final.

Preprocesamiento

El pipeline de preprocesamiento incluyó dos etapas principales, siempre ajustadas sobre el train y aplicadas al val y test:

- Codificación de variables categóricas con estrategia diferenciada según cardinalidad: One-Hot Encoding para Ship Mode, Segment, Region, Category y Sub-Category (baja cardinalidad, entre 2 y 17 valores únicos); Target Encoding con smoothing=10 para State (49 estados, cardinalidad media-alta que haría ineficiente el OHE).
- Escalado con RobustScaler tanto para las features numéricas (X) como para la variable objetivo (y). El RobustScaler usa la mediana y el rango intercuartílico en lugar de la media y el desvío, siendo resistente a los outliers presentes en Sales y Profit. Escalar y permite que el modelo aprenda en un espacio normalizado; las predicciones se desnormalizan con `inverse_transform` para obtener los valores en dólares.

Diseño e Implementación de la Red Neuronal

Se diseñó un MLP implementado en PyTorch con arquitectura decreciente de 3 capas ocultas (128 → 64 → 32) y salida lineal de 1 neurona. Cada capa oculta sigue el patrón Linear → BatchNorm1d → ReLU → Dropout(0.1). La capa de salida no tiene función de activación, permitiendo predicciones en cualquier rango real, incluyendo valores negativos.

Selección Experimental de la Función de Pérdida

Antes de entrenar el modelo final, se realizó un experimento controlado comparando tres funciones de pérdida candidatas (MSE, MAE y Huber Loss con $\delta=5$) entrenando un modelo rápido de 50 épocas con cada una y evaluando el MAE resultante en dólares sobre el test:

- MSE: MAE de \$31.13 — penaliza fuertemente los outliers de Profit, orientando el modelo hacia esos casos extremos a costa del error en el rango central.
- MAE (L1Loss): MAE de \$40.04 — más robusta a outliers pero con gradiente constante, lo que puede hacer el entrenamiento inestable cerca del mínimo.
- Huber Loss ($\delta=5$): MAE de \$30.74 — obtuvo el menor MAE, combinando el comportamiento cuadrático de MSE para errores pequeños con el comportamiento lineal de MAE para errores grandes.

La Huber Loss fue seleccionada como función de pérdida del modelo final, validando empíricamente su superioridad para este problema con outliers.

Entrenamiento

El modelo final fue entrenado con Early Stopping (paciencia de 20 épocas sobre el loss de validación) y hasta un máximo de 300 épocas con batch size de 64. El optimizador Adam ($\text{lr}=1\text{e-}3$, $\text{weight_decay}=1\text{e-}4$) se combinó con ReduceLROnPlateau (factor=0.5, paciencia=10 épocas) para reducir el learning rate cuando el loss de validación se estancaba. Los pesos del mejor epoch (menor loss de validación) se restauraron al finalizar.

Evaluación

La evaluación final sobre el conjunto de test en escala de dólares (tras `inverse_transform`) produjo los siguientes resultados: MSE de \$18.493, RMSE de \$135.99, MAE de \$25.67 y R^2 de 0.6186. El gráfico de Real vs Predicho mostró buena alineación en el rango central de Profit. El análisis de residuos confirmó que los errores son mayores en los valores extremos: el modelo subestima las pérdidas grandes y sobreestima las ganancias muy altas, comportamiento consistente con el uso de Huber Loss que prioriza el rango de mayor frecuencia.

Decisiones de Diseño

- RobustScaler en lugar de StandardScaler

El StandardScaler calcula la media y el desvío estándar, ambas estadísticas muy sensibles a valores extremos. Dado que Sales y Profit presentan outliers significativos (grandes ventas a empresas, pérdidas por devoluciones), el StandardScaler hubiera producido un escalado distorsionado donde los valores típicos quedarían comprimidos en un rango muy estrecho. El RobustScaler, al basarse en la mediana y el rango intercuartílico (estadísticas robustas a outliers), garantiza un escalado adecuado para la mayoría de las muestras sin verse arrastrado por los extremos.

Igualmente importante fue escalar también la variable objetivo (y). Entrenar el modelo con valores de Profit en dólares sin escalar implica que el modelo debe predecir valores en el rango $[-6.000, +8.000]$, lo que genera gradientes de magnitudes muy distintas y dificulta la convergencia. Escalar y al espacio del RobustScaler y luego invertir la transformación en evaluación produce predicciones equivalentes pero con un entrenamiento numéricamente más estable.

- Huber Loss con $\text{delta}=5$

La función de pérdida fue seleccionada de forma empírica comparando MSE, MAE y Huber Loss. La Huber Loss con $\text{delta}=5$ resultó ganadora porque combina las fortalezas de ambas alternativas: se comporta cuadráticamente (como MSE) para errores menores a 5 unidades escaladas, proporcionando gradientes suaves cerca del mínimo; y linealmente (como MAE) para errores mayores, siendo robusta ante los outliers de Profit sin ignorarlos completamente. El parámetro $\text{delta}=5$ fue elegido como un valor moderado que equilibra la sensibilidad a errores grandes con la estabilidad del entrenamiento. La validación empírica ($\text{MAE}_{\text{Huber}} = \30.74 vs $\text{MAE}_{\text{MSE}} = \31.13 vs $\text{MAE}_{\text{L1}} = \40.04) confirmó que esta elección era la más adecuada para el problema.

- Target Encoding para State con Smoothing

Se adoptó una estrategia de codificación diferenciada según la cardinalidad de cada variable categórica. Para las variables de baja cardinalidad (Ship Mode con 4 valores, Segment con 3, Region con 4, Category con 3, Sub-Category con 17) se aplicó One-Hot Encoding, que es la

técnica estándar y no introduce orden artificial entre categorías. Para State (49 estados), el OHE hubiera generado 49 columnas binarias muy dispersas, con muchos estados subrepresentados. El Target Encoding resuelve esto reemplazando cada estado por el promedio de Profit de ese estado en el train, comprimiendo toda la información geográfica en una sola columna numérica densa y predictivamente relevante. El parámetro $\text{smoothing}=10$ atenúa el encoding hacia la media global para estados con pocas muestras, evitando overfitting en estados poco frecuentes.

- Salida Lineal sin Activación

La capa de salida del MLP tiene 1 neurona sin función de activación (salida lineal). Esta es la decisión arquitectónica fundamental que distingue a una red de regresión de una de clasificación: en clasificación binaria se usa sigmoide (para acotar la salida en $[0, 1]$) y en clasificación multiclase se usa Softmax (para producir una distribución de probabilidad). En regresión, cualquier función de activación que acote la salida (sigmoide, tanh) limitaría el rango de valores predichos e impediría al modelo predecir Profit muy negativo o muy positivo. La salida lineal deja que el modelo prediga cualquier valor real, compatible con el rango ilimitado de la variable objetivo.

- Eliminación de la Transformación Logarítmica sobre y

Durante el desarrollo se exploró aplicar una transformación log-sign sobre Profit antes del escalado, con la intuición de que comprimiría los outliers y facilitaría el aprendizaje. Sin embargo, esta transformación resultó contraproducente: al invertirla en la evaluación (mediante la función exponencial), los errores del espacio transformado se amplificaban desproporcionadamente en el espacio original, llevando el R^2 a valores negativos (alrededor de -4.2). Eliminar la transformación logarítmica y escalar directamente con RobustScaler fue la decisión que llevó el R^2 de -4.2 a 0.62, representando la mejora más impactante del proceso de desarrollo.

Desafíos Encontrados

- Alta Variabilidad Inherente de la Variable Objetivo: el Profit de una transacción comercial depende de muchos factores no observables en el dataset: costos internos del producto, negociaciones individuales con clientes, descuentos especiales no registrados, y condiciones de mercado. Esto impone un techo teórico de R^2 que ningún modelo puede superar independientemente de su complejidad.
- Manejo de Outliers: la presencia de outliers significativos en Sales y Profit planteó un dilema metodológico. Eliminarlos hubiera producido un dataset más limpio para entrenar, pero el modelo resultante sería incapaz de predecir correctamente las transacciones de alto valor. Mantenerlos sin ningún tratamiento especial hubiera sesgado el MSE hacia esos casos extremos. La solución adoptada fue conservarlos pero utilizar RobustScaler (para el escalado) y Huber Loss (para el entrenamiento), dos herramientas diseñadas específicamente para ser robustas ante valores extremos sin ignorarlos por completo.
- Dificultad para Predecir Valores Extremos de Profit: el análisis de residuos reveló un patrón claro: el modelo predice bien en el rango central de Profit (valores entre -100 y +500 dólares) pero subestima las pérdidas grandes y sobreestima las ganancias muy altas. Esto es un comportamiento esperado pero difícil de eliminar: los valores extremos

son, por definición, los menos representados en el dataset de entrenamiento, y la Huber Loss reduce explícitamente su peso en el cómputo del gradiente. Los 5 casos con mayor error absoluto corresponden exactamente a transacciones con Profit muy extremo, que son atípicas y difíciles de predecir independientemente del modelo utilizado.

Conclusiones

En este problema se evidenciaron las diferencias clave entre redes de regresión y clasificación, destacando cambios en la capa de salida, funciones de pérdida, métricas y la necesidad de escalar la variable objetivo en regresión. Se demostró que la elección de la función de pérdida es crítica en presencia de outliers, siendo Huber la opción más equilibrada frente a MSE y MAE. También se resaltó el riesgo de aplicar transformaciones sobre la variable objetivo sin evaluar su impacto en el espacio original. El uso de RobustScaler resultó adecuado para datos de negocio con valores extremos, mientras que se reconocieron las limitaciones de los MLP frente a modelos de gradient boosting en datos tabulares. Finalmente, la comparación con los problemas anteriores permitió consolidar una visión integral: cada tipo de tarea requiere decisiones de diseño específicas, y se propusieron mejoras futuras como el uso de XGBoost, embeddings para variables categóricas y validación cruzada para obtener métricas más robustas.